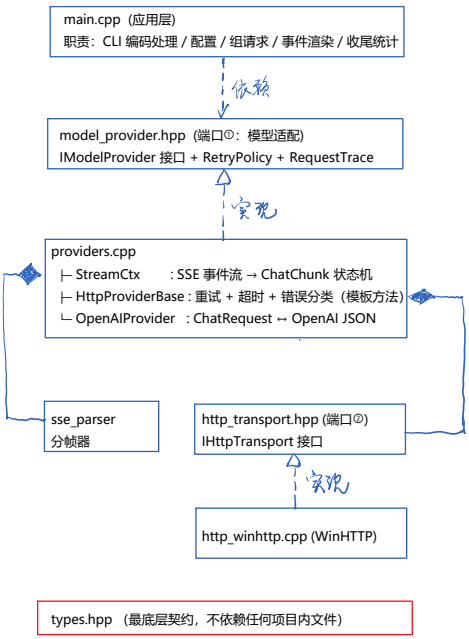
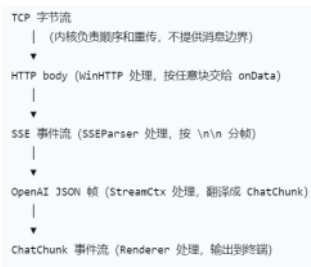
L3-v0

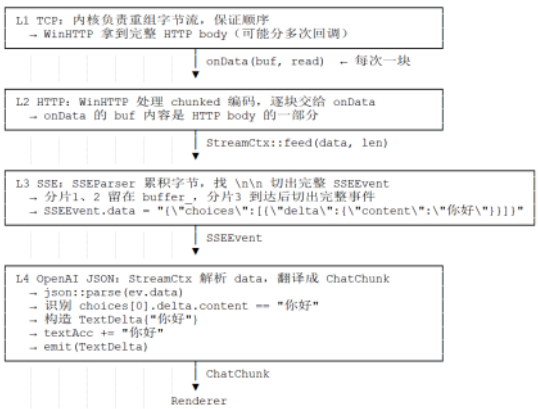
2026年10月12日 11:21

1. 文件架构：



2. 包处理各异：





3. 包处理核心组件



StreamCtx 的"起承转结"：整条流式管道的翻译中枢

StreamCtx 是 providers.cpp 内部定义的一个局部状态机，它把"原始字节流"翻译成"语义事件流"。用"起承转结"来概括，是因为它恰好承担了四个角色：起（接收字节）、承（跨块累积状态）、转（协议翻译）、结（收尾汇总）。

为什么需要 StreamCtx？它解决的根本矛盾

上游（传输层）看到的是：任意切分的字节块。

下游（Renderer）看到的是：结构化的语义事件。

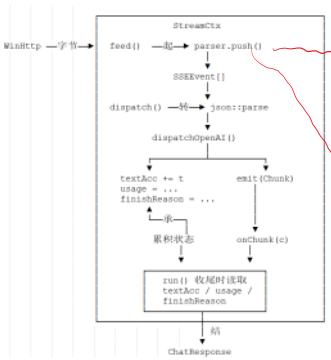
这两者之间有三重鸿沟：

鸿沟

上游视角

下游视角

边界
一个 TCP 分片可能含半个事件、一个半事件
每个 ChatChunk 是完整的语义单位
协议
OpenAI 的 JSON 帧格式
与厂商无关的 ChatChunk variant
状态
无状态，每次回调只是一个 char*
需要累积（正文、usage、finish_reason）
StreamCtx 就是跨越这三重鸿沟的唯一桥梁。它是管道上唯一同时知道"字节长什么样"和"语义长什么样"的组件。




```
struct StreamCtx {
    SSEParser parser;                          // 跨块缓冲
    std::function<bool(const ChatChunk&)> onChunk; // 返回 false = 取消
    std::string  textAcc;                      // 累积正文（最终答案）
    Usage        usage;                        // 累积用量
    FinishReason finishReason = FinishReason::Stop;
};
```

SSEParser 的状态机



4. 代码实现

用户输入 "用一句话解释 SSE"
 |
 ▼ main.cpp
winUtf8Args() → sanitizeUtf8(task)
 |
 ▼
组装 ChatRequest {system, messages:[user]}
 |
 ▼
provider->stream(req, [renderer](c) { renderer(c); return true; })
 |
 ▼ OpenAIProvider::run()
toRequest() → JSON
body["stream_options"] = {include_usage: true}
 |
 ▼ HttpProviderBase::execute()
HttpRequestSpec { url, headers, body, timeouts }
 |
 ▼ WinHttpTransport::doRequest()
WinHttpOpen → Connect → OpenRequest → SendRequest → ReceiveResponse
 |
 ▼ 循环 WinHttpReadData
onData(buf, read) ——► StreamCtx::feed()
 |
 ▼ SSEParser::push()
按 \n\n 切块 → parseBlock() → SSEEvent
 |
 ▼ dispatchOpenAI()
 ┌──────────────────────────┐
 | usage? → UsageChunk |
 | content? → TextDelta |
 | reasoning_content? → ... |
 | finish_reason? → DoneChunk |
 └──────────────────────────┘
 |
 ▼ onChunk(c)
Renderer::operator()(c)
 |
 ▼ std::visit + overloaded
std::cout << ... << flush

```
struct Renderer {
    ......
    void operator()(const ChatChunk& c) {
        std::visit(overloaded{
            [&](const TextDelta& d) {
                if (!d.text.empty()) markFirstToken();
                std::cout << d.text << std::flush;
            },
            [&](const ThinkingDelta& d) {
                if (!d.text.empty()) markFirstToken();
                std::cout << "\033[2m" << d.text << "\033[0m" << std::flush;
            },
            [&](const ToolCallStart& s) {
                std::cerr << "\n[未识别: 收到工具调用 " << s.name
                          << "（工具在 L4 才引入）\n";
            },
            [&](const ToolCallDelta&) {},
            [&](const ToolCallDone&)  {},
            [&](const UsageChunk& u) { usage = u.usage; ++usageEvents; },
            [&](const ErrorChunk& e) {
                sawError = true;
                std::cerr << "\n[流内错误] " << e.message
                          << (e.retryable ? "（可重试）" : "") << "\n";
            },
            [&](const DoneChunk& d) { finish = d.reason; },
        }, c);
    }
```

OpenAIProvider::stream

```
ChatResponse stream(const ChatRequest& req,
                    const std::function<bool(const ChatChunk&)>& cb) override {
    StreamCtx ctx;                  // ← 栈上对象
    ctx.onChunk = cb;               // ← 保存上层 (main.cpp 的 Renderer) 回调
    return run(req, &ctx);          // ← 传指针下去
}

ChatResponse run(const ChatRequest& req, StreamCtx* sctx) {
    json body = toRequest(req, sctx != nullptr);
    if (sctx) body["stream_options"] = {{"include_usage", true}};

    std::vector<std::string> headers{"Authorization: Bearer " + cfg_.apiKey};
    auto r = execute(cfg_.baseUrl + "/chat/completions", body.dump(), headers, sctx);
    //                                                                  ↑
    //                                                          sctx 继续往下传
    ...
}
```

HttpProviderBase::execute

```
HttpResult execute(const std::string& url, const std::string& payload,
                   const std::vector<std::string>& extraHeaders,
                   StreamCtx* sctx) {
    HttpRequestSpec spec;
    spec.url = url;
    spec.body = payload;
    // ... 组装 headers ...

    for (int attempt = 0; attempt < cfg_.retry.maxAttempts; ++attempt) {
        HttpResponseInfo info;

        if (sctx) {
            // *** 这里创建 Lambda ***
            info = transport_->postStream(spec, [sctx](const char* d, std::size_t n) {
                return sctx->feed(d, n);
            });
            //                  ↑
```

StreamCtx

```
struct StreamCtx {
    SSEParser parser;

    std::function<bool(const ChatChunk&)> onChunk;

    bool emit(const ChatChunk& c) { return onChunk ? onChunk(c) : true; }
};

bool dispatchOpenAI(const json& j) {
    bool keep = true;

    // ---- ① usage ----
    if (j.contains("usage") && !j["usage"].is_null()) {
        usage.inputTokens  = u.value("prompt_tokens", usage.inputTokens);
        usage.outputTokens = u.value("completion_tokens", usage.outputTokens);
        keep = emit(UsageChunk{usage}) && keep;          // ← 产生 ChatChunk #1
    }

    if (!j.contains("choices") || j["choices"].empty()) return keep;

    const auto& ch = j["choices"][0];
    const auto  d  = ch.value("delta", json::object());

    // ---- ② 正文 / 思考 ----
```

```
bool dispatch(const SSEEvent& ev) {
    if (ev.data.empty()) return true;                    // 空 data 直接跳过
    // 容错解析：坏帧直接跳过，**不抛** throw** —— 一个坏帧不该毁掉整个会话。
    json j = json::parse(ev.data, nullptr, false);       // ← 非抛出版本
    if (j.is_discarded()) return true;                   // 解析失败：静默跳过
    return dispatchOpenAI(j);
}

bool feed(const char* data, std::size_t len) {
    for (auto& ev : parser.push(std::string_view(data, len))) {
        if (ev.isDone()) return false;       // [DONE] 或 message_stop
        if (!dispatch(ev)) return false;     // 上层取消
    }
    return true;
}
```

WinHttpTransport::postStream

```
HttpResponseInfo postStream(
    const HttpRequestSpec& req,
    const std::function<bool(const char* data, size_t len)>& onData) override {
    return doRequest(req, onData);   // ← 只是转发
```



从 SSE 分帧到 Renderer 调用：

```
SSEParser::push(chunk) → vector<SSEEvent>
                          │
                          ▼  StreamCtx::feed 遍历
                    dispatch(ev)
                          │ ① JSON 解析
                          ▼
                    json j = json::parse(ev.data)
                          │
                          ▼
                    dispatchOpenAI(j)
                          │ ② 构造 ChatChunk
                          │    (一个 SSEEvent 可能产生 0~N 个 ChatChunk)
                          ▼
                    emit(ChatChunk)
                          │ ③ 调用 onChunk
                          ▼
                    onChunk(c)   ← main.cpp 传入的 lambda
                          │
                          ▼
                    renderer(c)  ← 捕获的 Renderer 对象
                          │
                          ▼
                    Renderer::operator()(c)
                          │ ④ std::visit 分发
                          ▼
                    overloaded{...} 中匹配的分支
                          │
                          ▼
                    std::cout << ...
```

完整时序示例

假设服务端依次发送以下 4 个 SSE 帧（每帧之间可能被 TCP 任意切分）：

帧1: {"choices":[{"delta":{"role":"assistant","content":""}}]}
帧2: {"choices":[{"delta":{"content":"SSE 是"}}]}
帧3: {"choices":[{"delta":{"content":"服务器推送。"}}]}
帧4: {"choices":[{"delta":{},"finish_reason":"stop"}],"usage":{"prompt_tokens":12,"completion_tokens":6}}
帧5: data: [DONE]

步骤	SSEEvent.data	dispatchOpenAI产生	onChunk 调用	Renderer 分支
帧1	{...content:""}	TextDelta("")	1 次	TextDelta：空串，不标记首 token，打印空
帧2	{...content:"SSE 是"}	TextDelta("SSE 是")	1 次	TextDelta：标记首 token，打印 "SSE 是"
帧3	{...content:"服务器推送。"}	TextDelta("服务器推送。")	1 次	TextDelta：打印 "服务器推送。"
帧4	{...finish_reason:"stop", usage:...}	UsageChunk + DoneChunk{Stop}	2 次	UsageChunk 保存用量；DoneChunk 保存 finish
帧5	[DONE]	—	—	（isDone() 直接 return false，中止传输）